

SOFTWARE REQUIREMENTS SPECIFICATION

Blockchain-Based Digital Certification System

Version 2.0 — With System Diagrams + Cookie Auth + 2FA

Swami Keshvanand Institute of Technology (SKIT), Jaipur
Department of Computer Science & Engineering | April 2026

Table of Contents

I. Introduction

I.1 Purpose

This Software Requirements Specification (SRS) v2.0 defines complete functional and non-functional requirements for the Blockchain-Based Digital Certification System (BDCS). This version incorporates updated authentication design (HttpOnly Cookie-based sessions with TOTP-based Two-Factor Authentication) and includes full system diagrams for all major workflows.

This document is the primary reference for the development team, project evaluators, guide faculty, and conference reviewers throughout the 10-month project lifecycle.

I.2 Project Scope

BDCS is a full-stack web platform enabling vocational institutions to issue tamper-proof digital certificates stored cryptographically on the Polygon blockchain. The system provides instant public verification, a personal credential wallet for learners, institution management, and an admin approval panel.

I.3 Problem Statement

India's vocational education sector issues millions of certificates annually through systems that suffer from:

- Certificate forgery and duplication undermining employer trust
- Slow, manual verification delaying hiring and enrollment
- No interoperability between credential systems and institutions
- Learners permanently losing access to credentials over time
- No tamper-proof audit trail for issuance or revocation events

I.4 Document Conventions

- Must Have — Core requirement; system is incomplete without it
- Should Have — High value; include unless time-constrained
- FR-XX — Functional Requirement | NFR-XX — Non-Functional Requirement

2. Overall System Description

2.1 System Architecture Overview

BDCS follows a strict 4-layer architecture. The frontend never communicates with the blockchain directly — all blockchain interactions are mediated exclusively by the FastAPI backend layer.

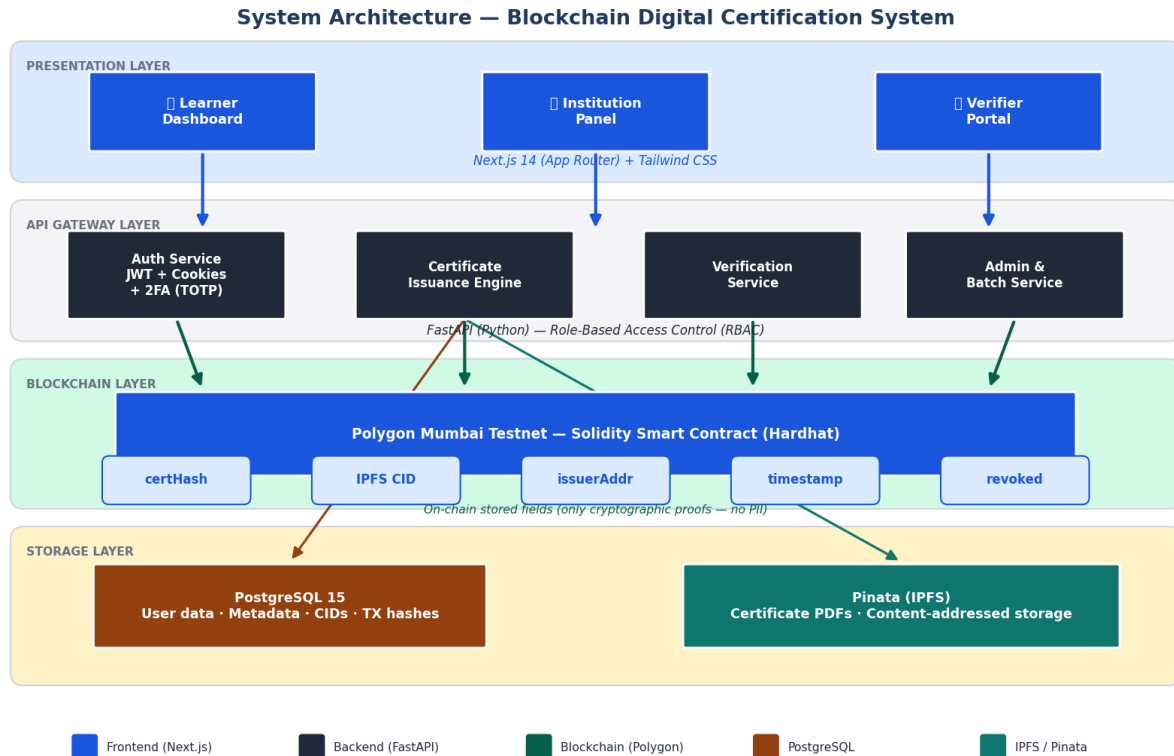


Figure 1: System Architecture — 4-Layer Stack (Next.js · FastAPI · Polygon · PostgreSQL/IPFS)

2.2 Technology Stack

Layer	Technology	Version	Purpose
Frontend	Next.js	14 (App Router)	SSR UI + API routes + role dashboards
Styling	Tailwind CSS	3.x	Responsive design
Backend	FastAPI	0.110+	REST APIs + business logic
Auth	Cookie (HttpOnly)	—	Secure session management
2FA	pyotp (TOTP)	—	Google Authenticator-style 2FA
Password	bcrypt	—	Password hashing
Blockchain	Polygon Mumbai	Testnet	Immutable certificate records
Smart Contracts	Solidity	^0.8.20	On-chain certificate logic
Chain Bridge	web3.py	6.x	Python to blockchain calls
File Storage	Pinata (IPFS)	—	Decentralized PDF storage

Database	PostgreSQL	I5+	Metadata + user data
ORM	SQLAlchemy	2.x	DB abstraction
PDF Gen	ReportLab	—	Certificate PDF creation
QR Code	qrcode	—	Verification QR generation
Dev Tools	Hardhat	—	Smart contract test + deploy

2.3 Data Storage Strategy

A critical design principle is the strict separation of what is stored where. Personal data never touches the blockchain:

Privacy Rule Only cryptographic hashes (SHA-256) and IPFS CIDs are stored on-chain. All PII (names, emails, personal details) are stored exclusively in PostgreSQL, off-chain.

Data	Blockchain	PostgreSQL	IPFS (Pinata)
Certificate PDF file	Never	No (CID only)	Yes — stored here
SHA-256 hash of PDF	Yes — primary	Yes — backup	No
IPFS CID	Yes	Yes	N/A (is the address)
Issuer address / ID	Yes (address)	Yes (full details)	No
Timestamp + TX hash	Yes	Yes	No
Revocation status	Yes — on-chain	Yes — mirror	No
Learner personal data	NEVER	Yes — encrypted	No

3. Authentication & Security Design

3.1 Authentication Strategy — v2.0 Update

After reviewing security best practices, the authentication system has been upgraded from JWT tokens in localStorage to HttpOnly Cookie-based sessions combined with TOTP Two-Factor Authentication. This is the production-standard approach used in enterprise applications.

3.2 Cookie-Based vs JWT Comparison

Feature	localStorage JWT (Old Plan)	HttpOnly Cookie (v2.0)
XSS Attack Surface	Vulnerable — JS can steal token	Protected — JS cannot access cookie
CSRF Protection	Manual implementation needed	SameSite=Strict handles it automatically
2FA Integration	Complex state management	Clean — cookie set only after 2FA passes
Implementation Complexity	Moderate	Simple — browser manages cookie automatically
Industry Standard	Acceptable	Production-standard approach

3.3 Authentication + 2FA Flow

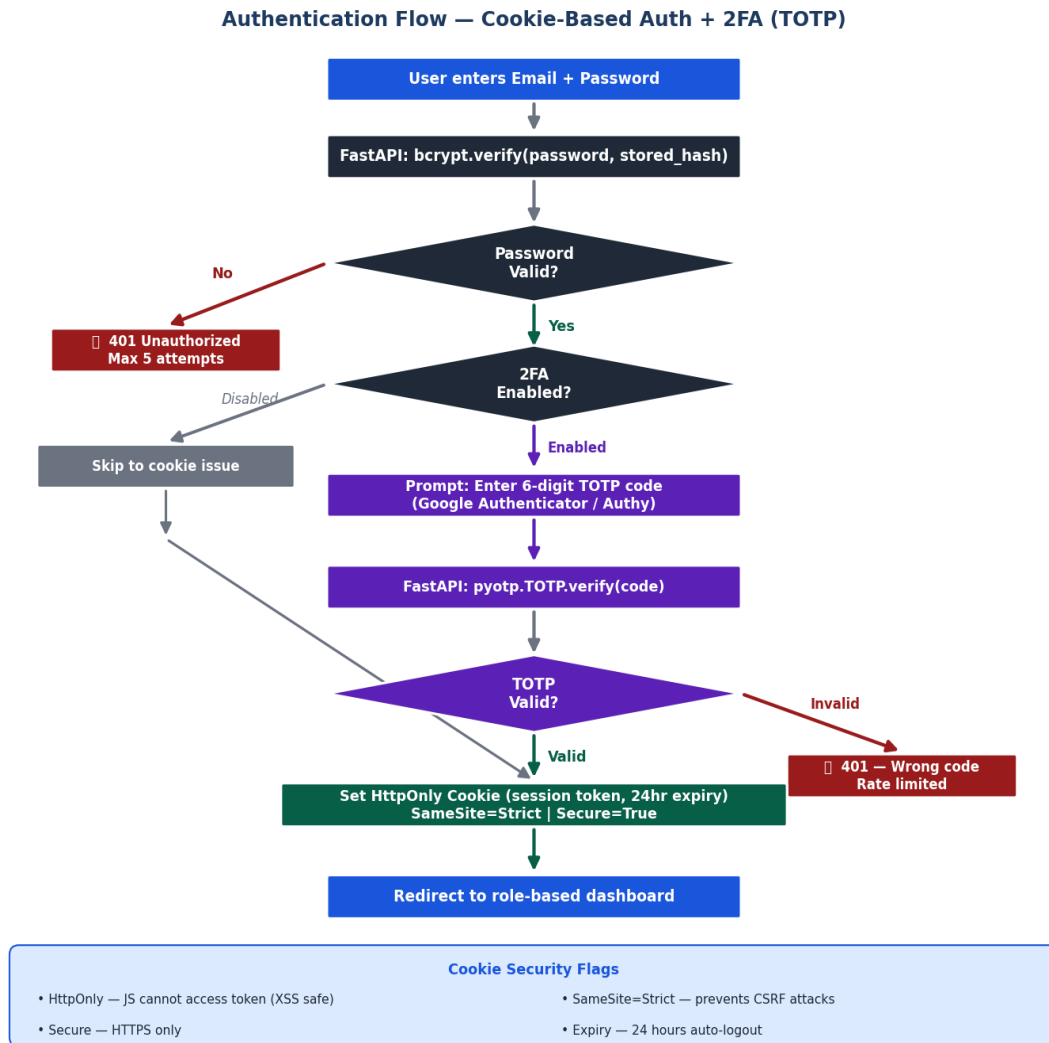


Figure 2: Complete Authentication Flow — bcrypt validation 'n TOTP 2FA 'n HttpOnly Cookie issuance

3.4 Cookie Security Configuration

All session cookies must be configured with these exact security flags in FastAPI:

- HttpOnly = True — JavaScript cannot read or steal the cookie (XSS protection)
- SameSite = Strict — Cookie not sent on cross-site requests (CSRF protection)
- Secure = True — Cookie only transmitted over HTTPS
- Max-Age = 86400 — 24-hour session expiry with automatic logout

3.5 Two-Factor Authentication (TOTP)

2FA is implemented using the pyotp library which generates Time-based One-Time Passwords (TOTP) compatible with Google Authenticator and Authy. The flow:

1. On first 2FA setup: server generates a TOTP secret via `pyotp.random_base32()`
2. Server returns a QR code image (`qrcode lib`) containing the `otpauth://` URI
3. User scans QR with their authenticator app
4. User confirms setup by entering the first valid 6-digit code
5. On every subsequent login: user enters fresh 6-digit code after password

6. Server validates: `pyotp.TOTP(secret).verify(code)` — codes expire every 30 seconds

Security Note 2FA is mandatory for Institution accounts and Admin accounts. It is optional but encouraged for Learner accounts. Verifier portal requires no authentication at all.

4. Functional Requirements

4.1 Authentication Module

FR ID	Module	Requirement	Priority
FR-01	Auth	User registration with role selection (Learner, Institution, Admin)	Must Have
FR-02	Auth	Password hashed using bcrypt before any storage	Must Have
FR-03	Auth	Session managed via HttpOnly Cookie (SameSite=Strict, Secure=True, 24h expiry)	Must Have
FR-04	Auth	Role-Based Access Control (RBAC) with 4 roles: Admin, Institution, Learner, Verifier	Must Have
FR-05	Auth	TOTP-based 2FA setup via QR code (pyotp + qrcode library)	Must Have
FR-06	Auth	2FA mandatory for Institution and Admin roles; optional for Learner	Must Have
FR-07	Auth	Rate limiting on login: max 5 failed attempts before 15-minute lockout	Should Have
FR-08	Auth	Password reset via email verification token	Should Have
FR-09	Auth	Admin approves or rejects institution registration requests with remarks	Must Have

4.2 Certificate Issuance Module

FR ID	Module	Requirement	Priority
FR-10	Issuance	Institution submits: learner name, course, grade, date, institution ID	Must Have
FR-11	Issuance	System generates formatted PDF using ReportLab	Must Have
FR-12	Issuance	System computes SHA-256 hash of PDF file	Must Have
FR-13	Issuance	PDF uploaded to Pinata IPFS; CID stored in DB and on blockchain	Must Have
FR-14	Issuance	Smart contract call via web3.py: stores hash + CID + issuer + timestamp on Polygon	Must Have
FR-15	Issuance	Unique Certificate ID (UUID) assigned to every certificate	Must Have
FR-16	Issuance	QR code generated and embedded in certificate PDF	Must Have
FR-17	Issuance	Batch issuance via CSV upload (up to 500 certs) processed asynchronously	Should Have
FR-18	Issuance	Learner notified by email on successful issuance	Should Have

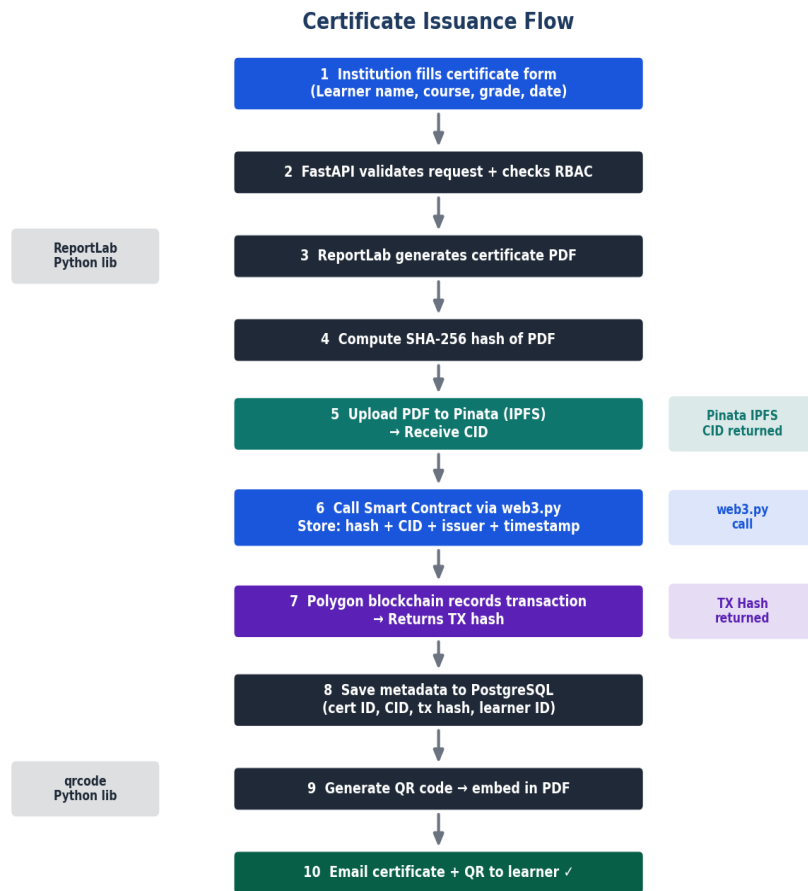


Figure 3: Certificate Issuance Flow — Institution → FastAPI → IPFS + Blockchain → Learner

4.3 Verification Module

FR ID	Module	Requirement	Priority
FR-19	Verification	Public portal verifies via QR code scan (no login required)	Must Have
FR-20	Verification	Public portal verifies via manual Certificate ID entry	Must Have
FR-21	Verification	Public portal verifies via PDF file upload (recompute + match hash)	Must Have
FR-22	Verification	Result shows: status (Valid/Revoked/Tampered), issuer, course, date, TX hash	Must Have
FR-23	Verification	Verifier requires no account — fully public endpoint	Must Have
FR-24	Verification	Backend fetches hash from blockchain and compares with recomputed hash	Must Have
FR-25	Verification	Verification response time under 2 seconds under normal conditions	Must Have

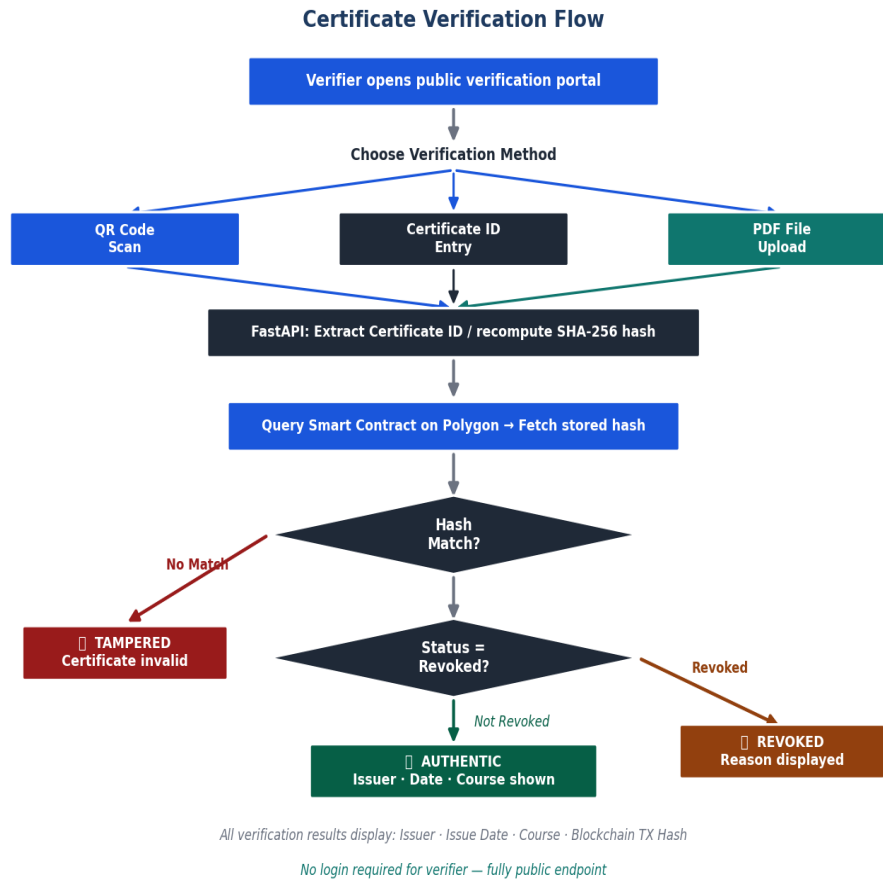


Figure 4: Verification Flow — 3 Methods 'n Blockchain Hash Match 'n Valid / Revoked / Tampered

4.4 Revocation Module

FR ID	Module	Requirement	Priority
FR-26	Revocation	Issuing institution can revoke any certificate it has issued	Must Have
FR-27	Revocation	Revocation updates the certificate status flag on the blockchain	Must Have
FR-28	Revocation	Revoked certificates show 'Revoked' status on all verification attempts	Must Have
FR-29	Revocation	Only original issuer address may call revoke on smart contract	Must Have
FR-30	Revocation	Institution provides a revocation reason (stored off-chain in PostgreSQL)	Should Have

4.5 Learner Dashboard (Credential Wallet)

FR ID	Module	Requirement	Priority
FR-31	Learner	View all certificates issued to learner's account	Must Have
FR-32	Learner	Download original PDF certificate	Must Have
FR-33	Learner	Copy or share unique public verification link per certificate	Must Have
FR-34	Learner	View certificate metadata: issuer, course, date, blockchain TX hash	Must Have
FR-35	Learner	Learner can enable/disable 2FA from account settings	Must Have

4.6 Institution Panel

FR ID	Module	Requirement	Priority
FR-36	Institution	Issue single certificates via form-based UI	Must Have
FR-37	Institution	Upload CSV for batch certificate issuance	Should Have
FR-38	Institution	View all issued certificates with filters (date, course, status)	Must Have
FR-39	Institution	Revoke certificates from panel with reason field	Must Have
FR-40	Institution	Dashboard analytics: total issued, revoked, verified count	Should Have

4.7 Admin Panel

FR ID	Module	Requirement	Priority
FR-41	Admin	View and action pending institution registration requests	Must Have
FR-42	Admin	Approve or reject institutions with remarks	Must Have
FR-43	Admin	View system-wide statistics (certs, institutions, verifications)	Must Have
FR-44	Admin	View logs of all issuance and revocation events	Should Have
FR-45	Admin	Deactivate institution accounts if needed	Should Have

4.8 Smart Contract Module

FR ID	Module	Requirement	Priority
FR-46	Blockchain	issueCertificate(certId, hash, ipfsCid, issuerAddress, timestamp)	Must Have
FR-47	Blockchain	verifyCertificate(certId) returns hash, issuer, timestamp, status	Must Have

FR-48	Blockchain	revokeCertificate(certId) callable only by original issuer address	Must Have
FR-49	Blockchain	Emit events: CertificateIssued, CertificateRevoked for indexing	Should Have
FR-50	Blockchain	Contract tested on Polygon Mumbai testnet before integration	Must Have

Smart Contract Architecture — Solidity on Polygon Mumbai

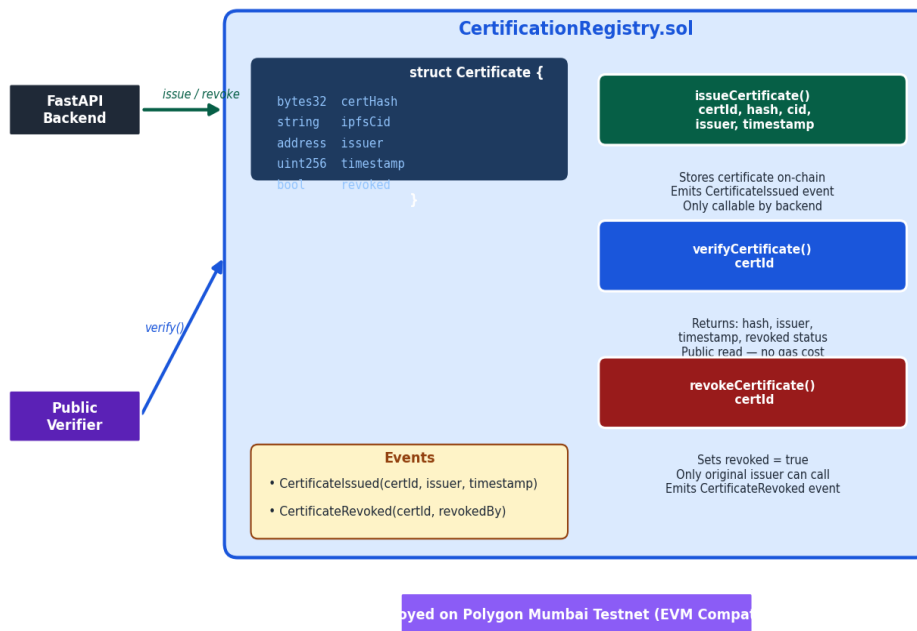


Figure 5: Smart Contract Architecture — CertificationRegistry.sol on Polygon Mumbai

5. Non-Functional Requirements

NFR ID	Category	Requirement	Metric
NFR-01	Performance	Verification API response under 2 seconds	< 2s
NFR-02	Scalability	Handle concurrent certificate issuance	1000+ certs/day
NFR-03	Security	All passwords bcrypt-hashed; sessions via HttpOnly cookie	bcrypt + cookie
NFR-04	2FA Security	TOTP codes expire every 30 seconds; brute force protected	pyotp TOTP
NFR-05	Availability	Minimal downtime during demo lifecycle	99% uptime
NFR-06	Data Integrity	PDF hash on blockchain must match stored PDF	SHA-256 100%
NFR-07	Privacy	PII never stored on blockchain — off-chain only	No PII on-chain
NFR-08	Usability	All primary flows without training	< 5 min onboard
NFR-09	Compliance	Indian IT Act 2000 + NCVET compatible	NCVET-ready

6. System Diagrams

6.1 Agile SDLC Model

The project follows the Agile Scrum methodology with 5 sprints of approximately 2 months each. Each sprint follows the standard Scrum cycle: Planning 'n Development 'n Integration 'n Testing 'n Review 'n Retrospective.

SDLC Model — Agile Scrum (10-Month, 5 Sprints of 2 Months Each)



Figure 6: Agile Scrum Model — 5 Sprints over 10 Months with Scrum Events and Artifacts

Scrum Events

- **Sprint Planning** — Start of each 2-month sprint; define backlog tasks for the sprint
- **Daily Standup** — 15-minute daily sync: what was done, what's next, any blockers
- **Sprint Review** — End of sprint demo to guide/evaluator; working software shown
- **Retrospective** — Team reflection: what worked, what to improve next sprint

Scrum Artifacts

- **Product Backlog** — All 50 functional requirements from this SRS, prioritized
- **Sprint Backlog** — Subset of requirements assigned to current sprint
- **Increment** — Working, tested, integrated software at end of each sprint

6.2 Entity Relationship Diagram

The PostgreSQL database schema consists of 6 primary entities with clearly defined relationships. All UUIDs are used as primary keys for global uniqueness across distributed systems.

Entity Relationship Diagram (ERD) — PostgreSQL Schema

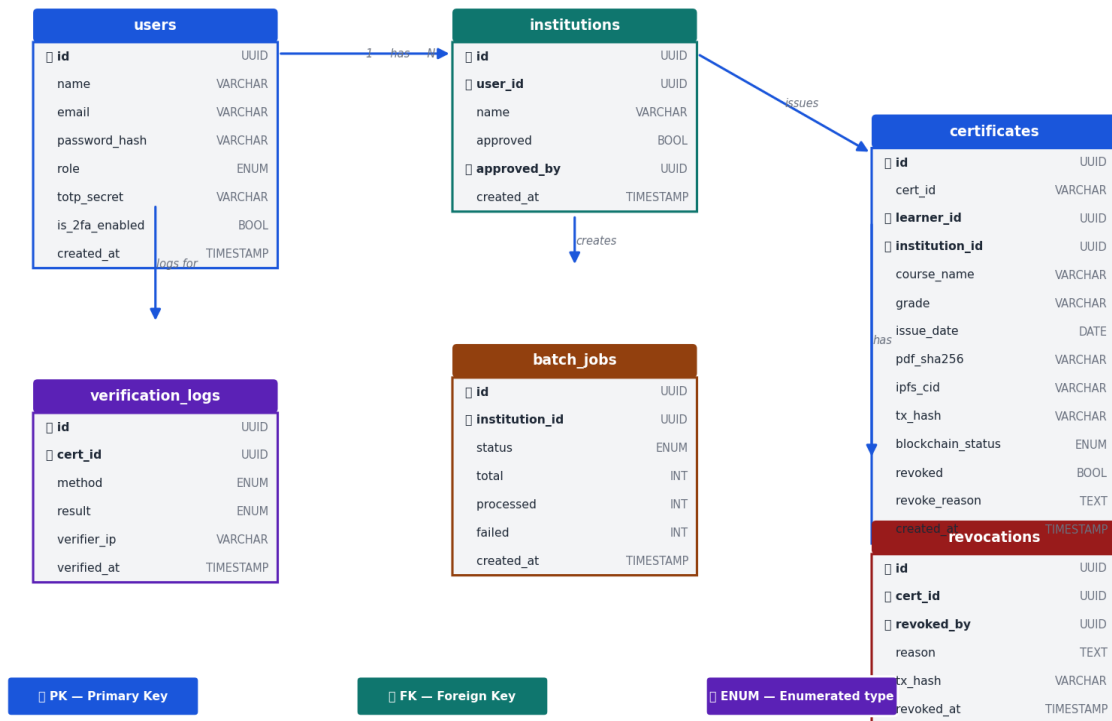


Figure 7: Entity Relationship Diagram — PostgreSQL Schema (6 entities, 8 tables)

Key Relationships

- *users* (1) 'n *institutions* (1): Each institution is linked to one user account
- *institutions* (1) 'n *certificates* (N): An institution issues many certificates
- *users* (1) 'n *certificates* (N): A learner receives many certificates
- *certificates* (1) 'n *verification_logs* (N): Each cert can be verified many times
- *certificates* (1) 'n *revocations* (1): A cert can have at most one revocation record
- *institutions* (1) 'n *batch_jobs* (N): An institution can run multiple batch imports

7. Project Timeline

Mo.	Phase	Deliverables	Milestone
1	Planning	SRS v2.0, DB schema, smart contract design, wireframes, Hardhat setup	Project kickoff
2	Blockchain	Solidity contracts coded, tested on Mumbai testnet, ABI locked	Contract deployed
3	Backend Core	FastAPI, Cookie Auth, TOTP 2FA, RBAC, PostgreSQL models	Auth system live
4	Issuance Engine	PDF gen, SHA-256 hash, Pinata upload, blockchain write, QR gen	Issuance working
5	Verification	3-method verify (QR/ID/file), revocation, public portal	Verification live
6	Frontend I	Next.js setup, auth pages, institution panel, admin dashboard	Institution UI done
7	Frontend II	Learner wallet, verifier portal, batch upload UI, 2FA setup page	All UIs done
8	Integration	Frontend + FastAPI + Blockchain end-to-end, full flow testing	Full flow working
9	QA & Security	Load testing, bug fixes, batch optimization, cookie security audit	Stable release
10	Launch	Deployment, documentation, demo video, conference prep	Project complete

7.1 Critical Path

These items must not slip — they block downstream work:

- Smart contract ABI locked by end of Month 2 — backend integration depends on stable ABI
- FastAPI issuance engine complete by Month 4 — frontend integration blocks on this
- Full integration sprint Month 8 — any slip compresses QA time in Months 9-10

Risk Mitigation If smart contract slips past Month 2, use a local Hardhat node mock so backend + frontend development can proceed in parallel without waiting for testnet deployment.

8. Team Responsibilities

Member	Role	Modules	Stack
Kartik (Lead)	Backend + Blockchain	FastAPI APIs, web3.py, Pinata, hashing, 2FA (pyotp)	Python, web3.py
Member 2	Frontend	Next.js, Tailwind, dashboards, QR scanner, cookie auth UI	Next.js, Tailwind
Member 3	Smart Contracts	Solidity, Hardhat, Polygon deploy, ABI management	Solidity, Hardhat
Member 4	DB + Auth + Admin	PostgreSQL, SQLAlchemy, JWT session, batch processing	FastAPI, SQL

8.1 Coordination Rules

- Weekly Monday sync — completed tasks, blockers, next week plan
- Git strategy: main 'n dev 'n feature/[member-name]/[feature] 'n PR to dev 'n PR to main
- API contract locked by end of Month 3 — before frontend development begins
- Smart contract ABI frozen end of Month 2 — no changes without team consensus
- Feature freeze at end of Month 6 — only bug fixes and polish after

9. Risk Analysis

Risk	Likelihood	Impact	Mitigation
Testnet instability	Medium	High	Fallback to local Hardhat node; mock mode for demo
IPFS pin expiry (Pinata free tier)	Medium	Medium	PostgreSQL as fallback; upgrade plan before demo
Team gap in Solidity	High	High	Assign dedicated owner; start contracts Month 1
Scope creep	High	Medium	SRS locked; no new features after Month 6
2FA misconfiguration	Low	High	Use pyotp test suite; QR setup page with verify step
Cookie security misconfiguration	Low	High	HttpOnly + SameSite=Strict enforced in FastAPI middleware
Performance under load	Low	High	Async batch via FastAPI BackgroundTasks

10. Out of Scope (v1.0)

- Digilocker / Skill India Digital / government API integrations
- Native mobile application (Android/iOS) — web is mobile-responsive
- Multilingual support — English only for v1.0
- On-chain storage of full certificate data — only hash and CID
- Real cryptocurrency / mainnet deployment
- SMS-based 2FA — TOTP app only (no external SMS API dependency)

10.1 Future Scope (v2.0 — Conference ideas)

- Digilocker and Academic Bank of Credits (ABC) integration
- ML-based anomaly detection for suspicious certificate patterns
- Employer verification REST API for HR system integration
- Multi-chain support (Ethereum + Polygon + Hyperledger Fabric)
- Biometric 2FA (WebAuthn / FIDO2) as alternative to TOTP

II. Glossary

Term	Definition
Blockchain	Distributed immutable ledger replicated across multiple nodes
Smart Contract	Self-executing code on blockchain enforcing business rules without intermediaries
SHA-256	Cryptographic hash producing a unique 256-bit fingerprint of any file
IPFS	InterPlanetary File System — peer-to-peer content-addressed file storage
CID	Content Identifier — the IPFS address of a file, derived from its hash
Pinata	IPFS pinning service ensuring files remain permanently accessible
HttpOnly Cookie	Session token stored in browser cookie inaccessible to JavaScript
TOTP	Time-based One-Time Password — 6-digit code regenerated every 30 seconds
pyotp	Python library for generating and validating TOTP 2FA codes
JWT	JSON Web Token — signed authentication token (not used in v2.0 for session)
RBAC	Role-Based Access Control — access management by user role
Polygon	EVM-compatible Layer 2 blockchain with fast, low-cost transactions
EVM	Ethereum Virtual Machine — runtime environment for Solidity contracts
Solidity	Statically-typed language for writing Ethereum/Polygon smart contracts
web3.py	Python library for interacting with EVM-compatible blockchains
FastAPI	High-performance Python web framework for REST APIs
SameSite=Strict	Cookie flag preventing transmission on cross-site requests (CSRF protection)
Revocation	Permanent invalidation of a certificate, recorded immutably on-chain
NCVET	National Council for Vocational Education and Training — India's credential regulator
SRS	Software Requirements Specification — this document

12. Approval and Sign-Off

This SRS v2.0 has been reviewed by all team members. Any changes post sign-off require written agreement from all members and must be logged as a v2.x revision.

Name	Role	Signature	Date
Kartik (Lead)	Backend + Blockchain		April 2026
Member 2	Frontend		April 2026
Member 3	Smart Contracts		April 2026
Member 4	DB + Auth		April 2026